



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

Selenium Framework and Real-Time Testing

JUL, 2025



Contents

- Introduction to Selenium Frameworks
- Page Object Model (POM)
- Page Factory implementation
- Data-driven testing using Excel and Flat Files in POM
- Drop Down List and its Methods
- Config Properties
- Log4j logging

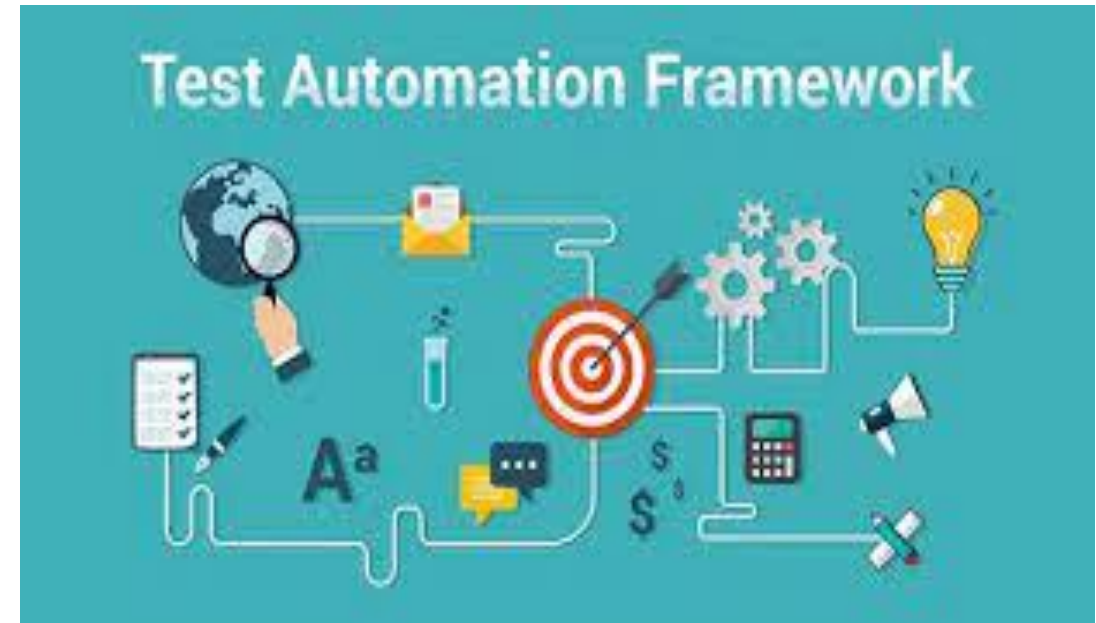
Introduction to Selenium Frameworks



What is Automation Framework?

Automation Framework

- Automation Framework is not a single tool or process, but it is a collection of tools and processes working together to support automated testing of any application.
- It integrates various functions like libraries, test data, and various reusable modules.



Why is Automation Framework?

Need of Automation Framework

- Framework supports automation testing as a technical implementation guideline.
- The automation framework not only offers the benefit of reusing the code in various scenarios, but it also helps the team to write down the test script in a standard format. Hence, the test automation framework handles all the issues.



Importance of Automation Framework

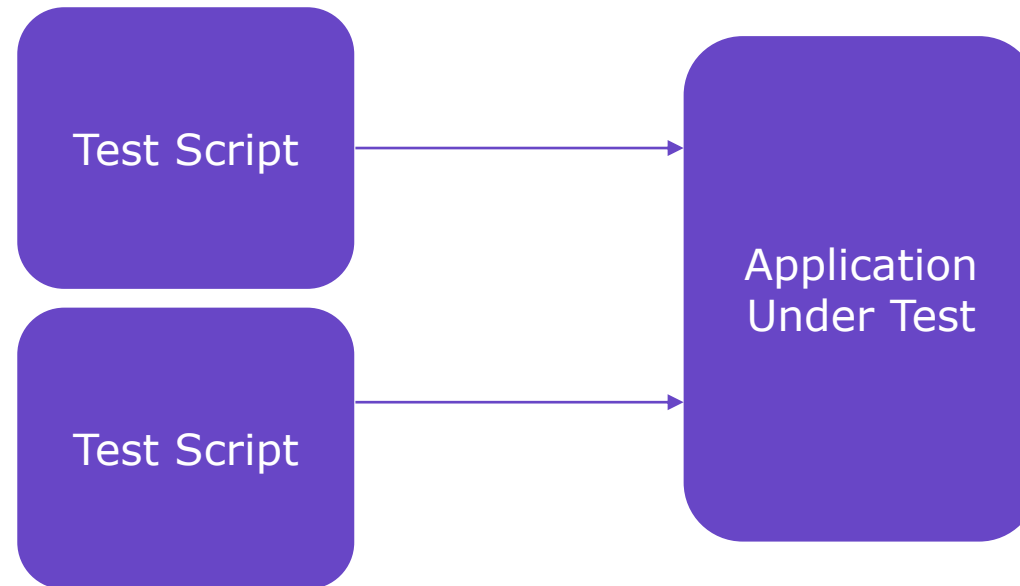
Importance of Automation Framework

- Maintain a well-defined strategy across the test suites.
- Enhance the speed at which the testing progresses.
- Maintaining the test code will be easy.
- The URL or application can be tested accurately.
- Continuous testing of code and delivery will be achieved.

Types of Automation Framework

Linear Driven Framework

- The linear Automation framework is commonly used in the testing of small applications.
- This framework is also called as a Record and playback framework.



Types of Automation Framework

Linear Driven Framework (Contd.)

Pros

- There is no need to write custom code, so expertise in test automation is not necessary

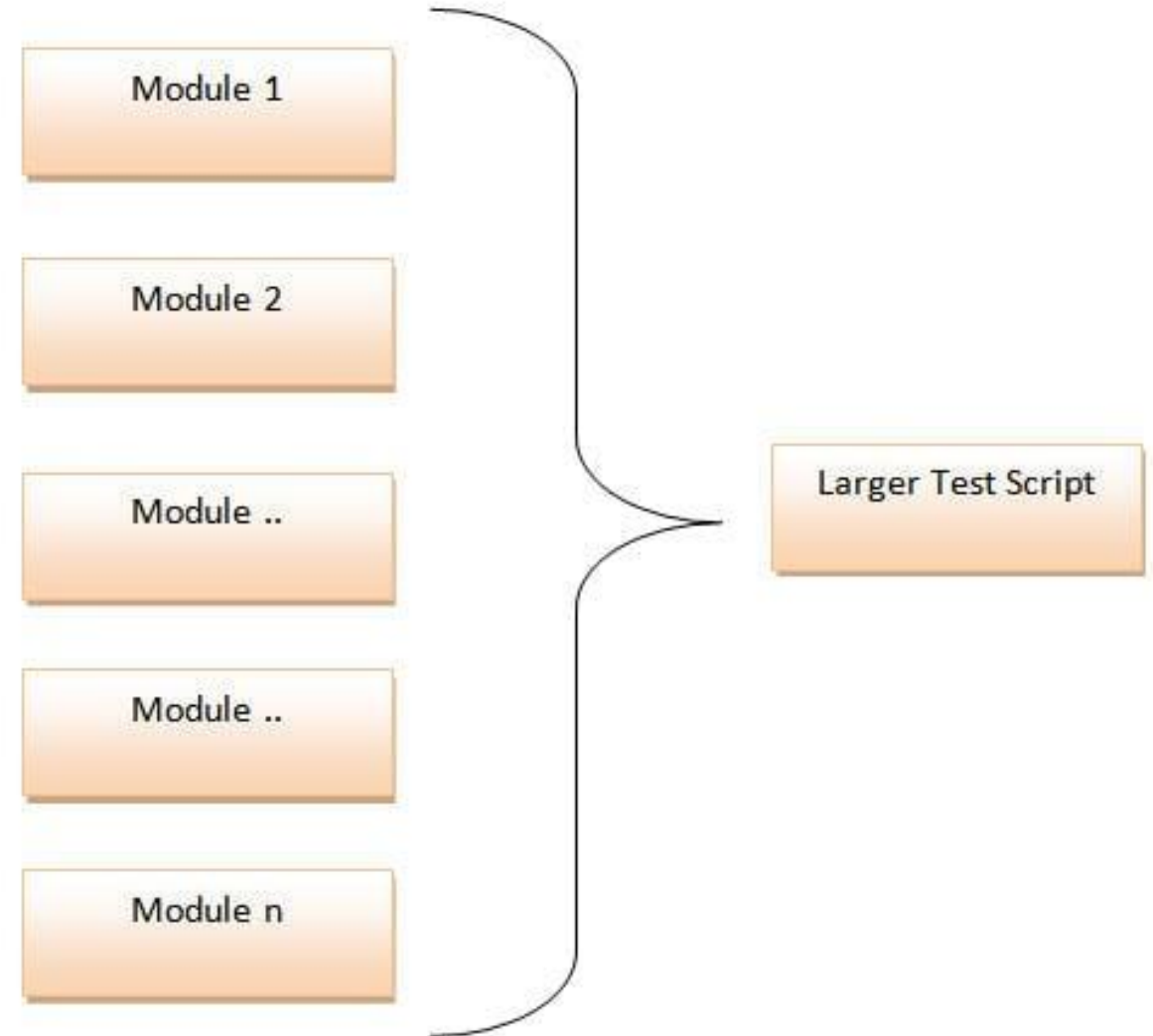
Cons

- The data is hardcoded in the test script; hence, the test cases cannot be re-run with multiple sets. You need to make some changes if the data is altered.

Types of Automation Framework

Modular Driven Framework

- In this Framework, the tester can create test scripts module wise by breaking down the whole application into smaller modules as per the client requirements and create test scripts individually



Types of Automation Framework

Modular Driven Framework (Contd.)

Pros

- Modular driven framework ensures the division of scripts that leads to easier maintenance and scalability. You can write independent test scripts.

Cons

- The modular driven framework requires additional time in analyzing the test cases and identifying reusable flows.

Types of Automation Framework

Behavior Driven Framework

- Behavior Driven Development framework is to create a platform, which allows every person, like Developers, Testers, business analyst, etc., to participate actively.
- It also increases collaboration between the tester and the developers on your project.



Types of Automation Framework

Behavior Driven Framework (Contd.)

Pros

- We can use non-technical, natural language to create test specifications on this behavior-driven testing.

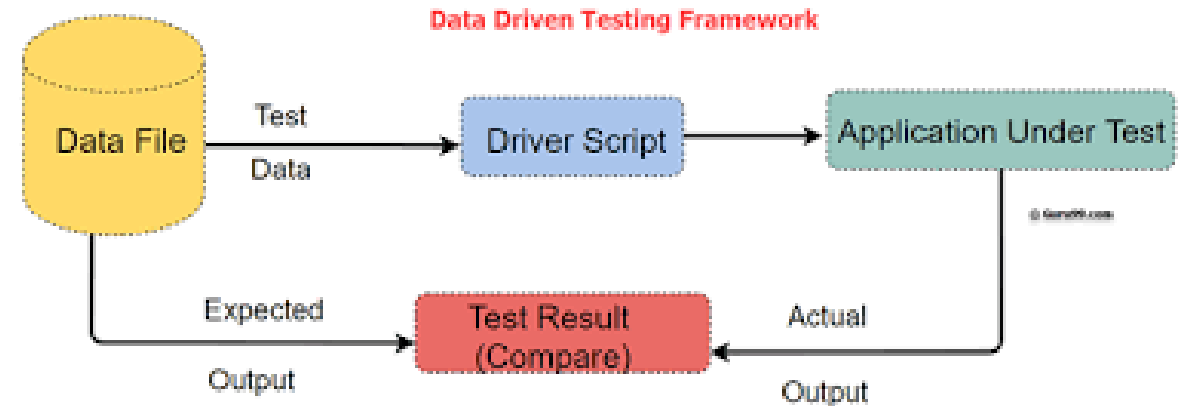
Cons

- To work with this framework, sufficient technical skills as well as prior experience in Test driven development is required.

Types of Automation Framework

Data Driven Framework

- Generally, Test Data is read from the external files like Excel Files, Text Files, CSV Files, ODBC Sources, DAO Objects and they are loaded into the variables inside the Test Script.
- The data-driven framework allows us to create test automation scripts by passing different sets of test data.



Types of Automation Framework

Data Driven Framework (Contd.)

Pros

- It reduces the number of scripts required. Hence, multiple scenarios can be tested in less code.

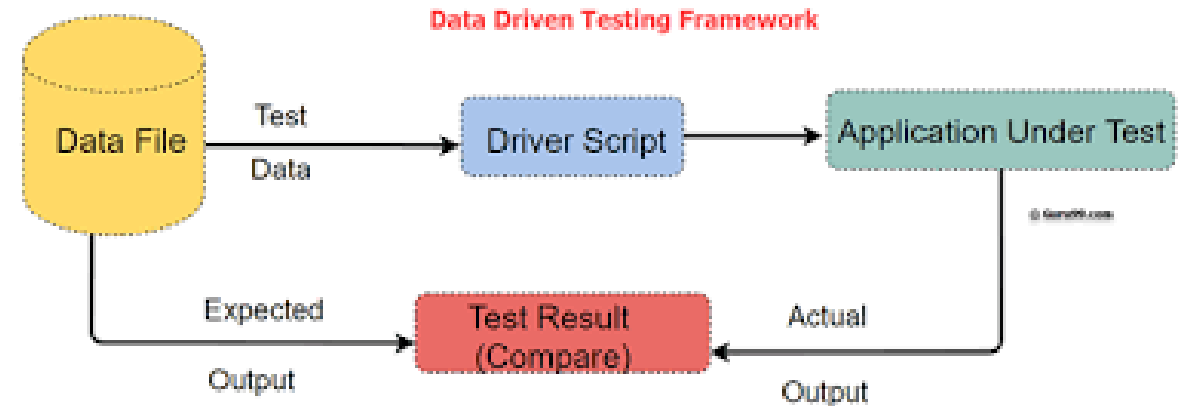
Cons

- You will need a highly experienced tester who should be proficient in various programming languages to completely utilize the design of this framework.

Types of Automation Framework

Keyword Driven Framework

- The Keyword-Driven Testing framework is also known as a table-driven testing.
- This framework is suitable only for small projects or applications
- The automation test scripts performed are based on the keywords specified in the excel sheet of the project.



Types of Automation Framework

Keyword Driven Framework (Contd.)

Pros

- A single keyword can be used across multiple test scripts, so the code is reusable.

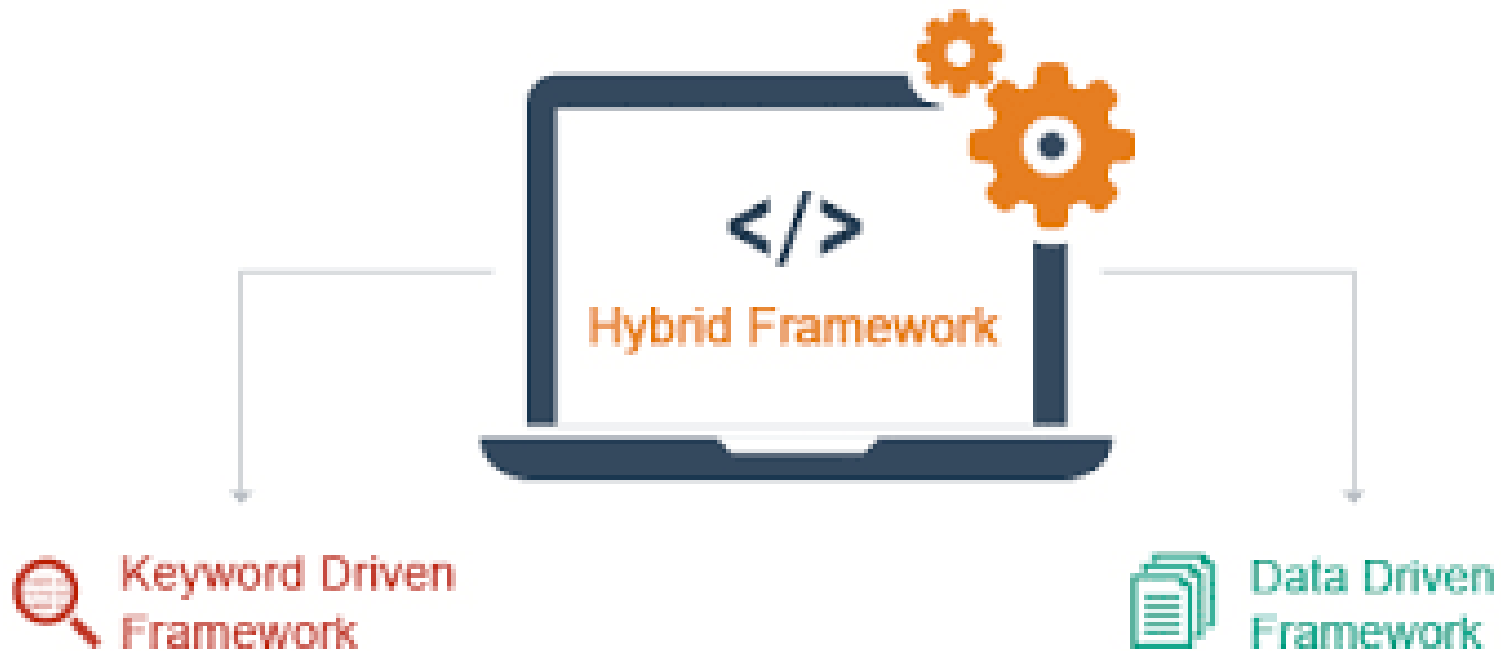
Cons

- The initial cost of setting up the framework is high, and it is time consuming & complex.

Types of Automation Framework

Hybrid Framework

- Hybrid Framework is used to combine the benefits of Keyword Driven and Data-Driven Frameworks



Types of Automation Framework

Hybrid Framework (Contd.)

Pros

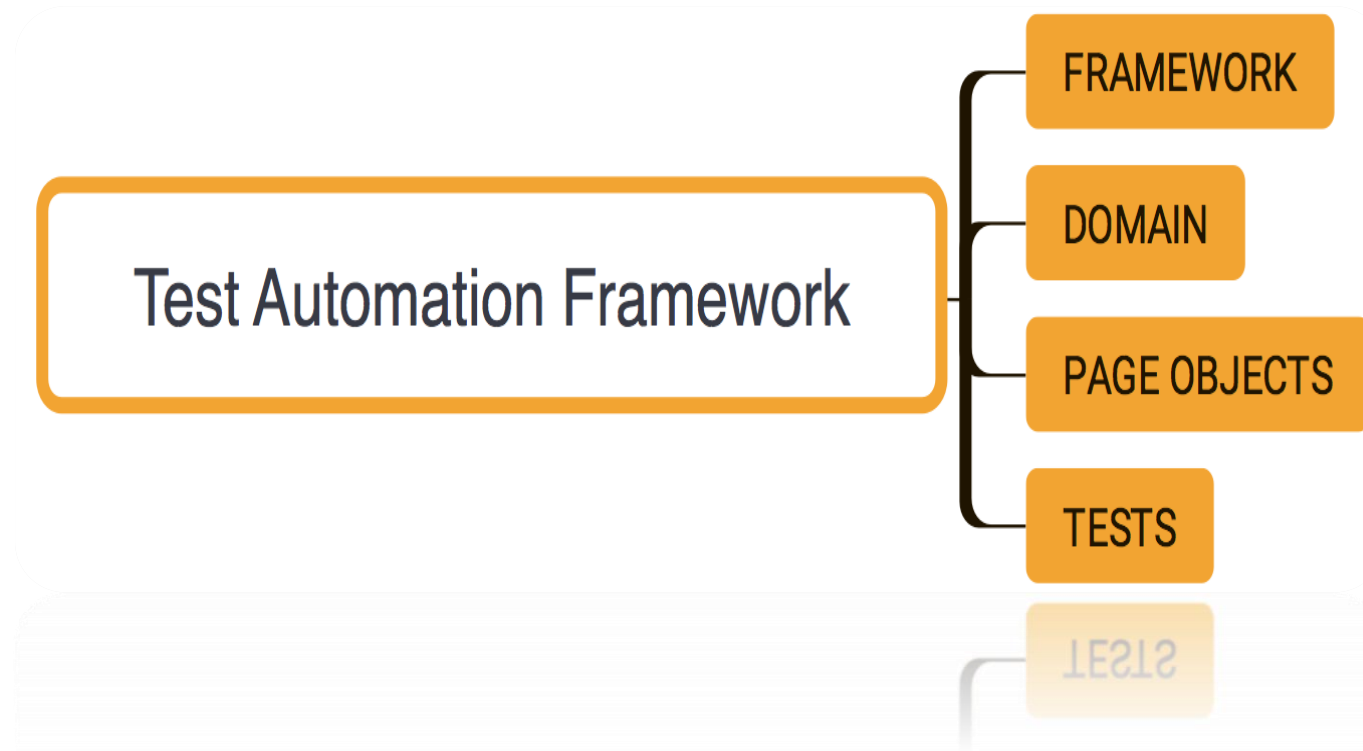
- This type leverages the advantages of all kinds of related frameworks.

Cons

- Tests are fully scripted in Hybrid Testing framework thus increase the automation effort

Test Automation Frameworks

Frameworks



How to build a maintainable Selenium framework?

**Choose a
programming
language**

**Choose a unit test
framework**

**Design the
framework
architecture**

**Build the
selenium core
component**

**Build the
selenium test
component**

**Choose the
reporting
mechanism**

**Decide how to
implement CI/CD**

**Integrate your
framework with
other tools**

How Do You Choose a Programming Language?

Usability

**Command-Line
Strength**

Simplicity

IDE Support

**Test Framework
Support**

**Industry
Adoption**

Choosing a Unit Test Framework

- Modular - The framework should be adaptable to change.
- Reusable – The commonly used methods or utilities should be kept in a common file which is easily accessible to all the scripts.
- Consistent – The test suite should be written in a consistent format by following the coding practices.
- Independent – The test scripts should be written in such a way that they are independent of each other.

Designing the Architecture of your Framework

- 1) Structure, Organize & Set Up Source Control
- 2) Familiarize yourself with the Application
- 3) Determine your testing environments & Gather Data
- 4) Set up a smoke test project
- 5) Create Utilities for on screen actions
- 6) Build and Manage Verifications
- 7) Set up your logging & reporting mechanism

Choose a Mechanism for Reporting

Reporting

- Unless we have a great reporting mechanism, it would not be easy to read the test results
- We would need to convert test results into insights, which can produce corrective results instantly.
- We can find a number of options to help to log your automated tests.
- In order to report this, We can use testing frameworks, including TestNG and JUnit.
- These are not in human-readable and need to be accessed using software such as CI or CD servers.

Choose a Mechanism for Reporting (Contd.)

Reporting

- But if we are using third-party libraries, including Allure or Extent Report, you would be able to create test result reports, which are in human-readable forms.
- Also, these would include screenshots and pie charts.
- These come with certain open-source reporting libraries, including Report NG for Java, which is an HTML reporting plugin for the TestNG framework.

Quiz



1) What is the primary purpose of using a test automation framework?

a) To slow down testing

b) To reduce manual effort and improve maintainability

c) To eliminate the need for coding

d) To remove test reports

Answer : Option b)

Quiz



2) Which of the following is NOT a type of Selenium automation framework?

a) Data-Driven Framework

b) Keyword-Driven Framework

c) Modular Framework

d) UI-Based Framework

Answer : Option d)

Quiz



3) What is the main characteristic of a Keyword-Driven Framework?

a) Uses object repositories only

b) Uses keywords in test scripts for actions

c) Requires manual execution

d) Does not support parameterization

Answer : Option b)

Quiz



4) In BDD, what are the keywords used to define test steps in Gherkin syntax?

a) WHEN, WHERE, WHY

b) BEGIN, THEN, END

c) GIVEN, WHEN, THEN

d) START, RUN, VERIFY

Answer : Option c)

Page Object Model (POM)

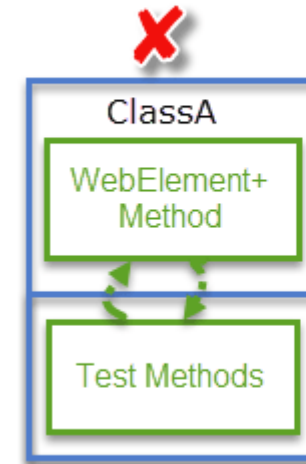


Page Object Model (POM)

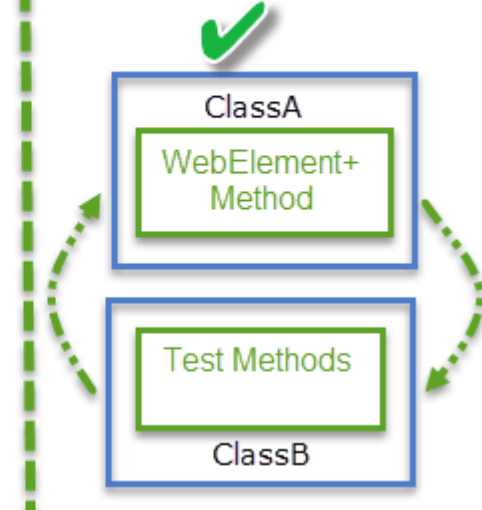
Introduction of Page Object Model

- Page Object Model (POM) is a design pattern, popularly used in test automation that creates Object Repository for web UI elements.
- The advantage is it reduces code duplication .
- In this model each web page of the application is considered as a separate class file

Non POM Structure



POM Based Structure



Page Object Model (POM)

Introduction of Page Object Model

- Page Object model is an object design pattern in Selenium, where web pages are represented as classes, and the various elements on the page are defined as variables in the class and all possible user interactions can then be implemented as methods in the class.
- The benefit is that if there is any change in the UI for the page, the tests themselves don't need to change, only the code within the page object needs to change.
- Subsequently all changes to support that new UI are located in one place.

Page Object Model (POM)

Introduction of Page Object Model (Contd.)

- The Page class will identify the web elements of that web page and also contains page methods which perform operations on that web elements
- **Page Object Model (POM)** is a design pattern, popularly used in test automation that creates Object Repository for web UI elements.
- The advantage of the model is that it reduces code duplication and improves test maintenance.

Page Object Model (POM)

Introduction of Page Object Model (Contd.)

- Under this model, for each web page in the application, there should be a corresponding Page Class.
- This **Page class will identify the WebElements of that web page** and also contains Page methods which perform operations on those WebElements.
- Name of these methods should be given as per the task they are performing, i.e., if a loader is waiting for the payment gateway to appear, POM method name can be `waitForPaymentScreenDisplay()`.

Page Object Model (POM)

Introduction of Page Object Model (Contd.)

- Under this model, for each web page in the application, there should be a corresponding Page Class.
- This **Page class will identify the WebElements of that web page** and also contains Page methods which perform operations on those WebElements.
- Name of these methods should be given as per the task they are performing, i.e., if a loader is waiting for the payment gateway to appear, POM method name can be `waitForPaymentScreenDisplay()`.

Page Object Model (POM)

Advantages of Page Object Model

1.Object Repository:

- You can create an Object Repository of the fields segmented page-wise.
- This as a result provides a Page Repository of the application as well.
- Each page will be defined as a java class.
- All the fields in the page will be defined in an interface as members. The class will then implement the interface.

2.Functional Encapsulation:

- All possible functionality or operations that can be performed on a page can be defined and contained within the same class created for each page.
- This allows for clear definition and scope of each page's functionality.

3. **Reusable** – If multiple test scripts use the same web elements, then we need not write code to handle the web element in every test script. Placing it in a separate page class makes it reusable by making it accessible by any test script.

Page Object Model (POM)

Advantages of Page Object Model

3.Low maintenance:

- Any User Interface changes can swiftly be implemented into the interface as well as class.

4.Programmer Friendly:

- Robust and more readable.
- The Object-oriented approach makes the framework programmer friendly.

5.Low Redundancy:

- Helps reduce duplication of code.
- If the architecture is correctly and sufficiently defined, the POM gets more done in less code.

6.Efficient & Scalable:

- Faster than other keyword-driven/data-driven approaches where Excel sheets are to be read/written.

Page Object Model (POM)

Disadvantages of Page Object Model

1.High Setup Time & Effort:

- Initial effort investment in development of Automation Framework is high.

2.Skilled labor:

- Testers not technically sound or aware of programming best practices are a nightmare in this case. Perhaps this is the biggest mistake to make, employing unskilled labor in hopes of training them during implementation.

3.Specific:

- Not a generic model. Automation Framework developed using POM approach is specific to the application. Unlike keyword-driven/data-driven frameworks, it is not a generic framework.
-

Page Object Model (POM)

Simple Automation Scenario – POM Implementation

1. Launch the Chrome/Firefox/Edge browser.
2. The OrangeHRM demo website opens in the browser.
3. Verify the Login Page
4. Enter username and Password and login to the demo site.
5. Verify the home page.
6. Close the browser.

```
OrangeHRMPOMBasic/  
├─ LoginPage.java  
├─ HomePage.java  
└─ OrangeHRMTest.java
```


LoginPage.java – POM class without PageFactory

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage {
    WebDriver driver;

    // Constructor
    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    // Locators
    By usernameField = By.name("username");
    By passwordField = By.name("password");
    By loginButton   = By.cssSelector("button[type='submit']");
    By loginPanel    = By.xpath("//h5[text()='Login']");
}
```

LoginPage.java – POM class without PageFactory

```
// Actions
public boolean isLoginPageDisplayed() {
    return driver.findElement(loginPanel).isDisplayed();
}

public void enterUsername(String username) {
    driver.findElement(usernameField).sendKeys(username);
}

public void enterPassword(String password) {
    driver.findElement(passwordField).sendKeys(password);
}

public void clickLogin() {
    driver.findElement(loginButton).click();
}
}
```

HomePage.java – POM class for Home Page

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class HomePage {
    WebDriver driver;

    public HomePage(WebDriver driver) {
        this.driver = driver;
    }

    // Locator for welcome message
    By dashboardHeader = By.xpath("//h6[text()='Dashboard']");

    public boolean isDashboardDisplayed() {
        return driver.findElement(dashboardHeader).isDisplayed();
    }
}
```

OrangeHRMTest.java – Test Class with main()

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class OrangeHRMTest {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();

        // Maximize and open the URL
        driver.manage().window().maximize();
        driver.get("https://opensource-demo.orangehrmlive.com/");
    }
}
```

OrangeHRMTest.java – Test Class with main()

```
// Create LoginPage object
LoginPage loginPage = new LoginPage(driver);

// Verify Login Page
if (loginPage.isLoginPageDisplayed()) {
    System.out.println("Login Page is displayed.");
} else {
    System.out.println("Login Page is NOT displayed.");
    driver.quit();
    return;
}

// Login to the application
loginPage.enterUsername("Admin");
loginPage.enterPassword("admin123");
loginPage.clickLogin();
```

OrangeHRMTest.java – Test Class with main()

```
// Create HomePage object
HomePage homePage = new HomePage(driver);

// Verify Home Page
if (homePage.isDashboardDisplayed()) {
    System.out.println("Login successful. Dashboard is displayed.");
} else {
    System.out.println("Login failed or Dashboard not found.");
}

// Close the browser
driver.quit();
}
```

Output in Console

```
Login Page is displayed.
Login successful. Dashboard is displayed.
```

Page Object Model (POM) with PageFactory

@FindBy Annotation

- The @FindBy annotation is used in Page Objects in Selenium tests to specify the object location strategy for a WebElement or a list of WebElements
- Using the PageFactory, these WebElements are usually initialized when a Page Object is created.
- The @FindBy annotation is used to locate one or more WebElements using a single criterion.

Syntax:

```
@FindBy(how = How.<strategy>, using = "locator")
```

WebElement element

Example:

```
@FindBy(id = "username")
```

WebElement usernameField;

Page Object Model (POM)

Simple Automation Scenario – POM Implementation using PageFactory

1. Launch the Chrome/Firefox/Edge browser.
2. The OrangeHRM demo website opens in the browser.
3. Verify the Login Page
4. Enter username and Password and login to the demo site.
5. Verify the home page.
6. Close the browser.

```
OrangeHRMPOMBasic/  
├─ LoginPage.java  
├─ HomePage.java  
└─ OrangeHRMTest.java
```


LoginPage.java – POM class with PageFactory

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;

public class LoginPage {
    WebDriver driver;

    // Constructor to initialize PageFactory
    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }

    // Elements with @FindBy
    @FindBy(name = "username")
    WebElement usernameField;

    @FindBy(name = "password")
    WebElement passwordField;
```

LoginPage.java – POM class without PageFactory

```
@FindBy(css = "button[type='submit']")
WebElement loginButton;

@FindBy(xpath = "//h5[text()='Login']")
WebElement loginPanel;
// Actions
public boolean isLoginPageDisplayed() {
    return loginPanel.isDisplayed();
}
public void enterUsername(String username) {
    usernameField.sendKeys(username);
}
public void enterPassword(String password) {
    passwordField.sendKeys(password);
}

public void clickLogin() {
    loginButton.click();
}
}
```

HomePage.java – POM class for Home Page

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;

public class HomePage {
    WebDriver driver;
    // Constructor
    public HomePage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }
    // Locator for Dashboard header
    @FindBy(xpath = "//h6[text()='Dashboard']")
    WebElement dashboardHeader;

    public boolean isDashboardDisplayed() {
        return dashboardHeader.isDisplayed();
    }
}
```

OrangeHRMTest.java – Test Class with main()

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class OrangeHRMTest {
    public static void main(String[] args) {
        // Setup ChromeDriver
        WebDriver driver = new ChromeDriver();

        // Maximize and navigate
        driver.manage().window().maximize();
        driver.get("https://opensource-demo.orangehrmlive.com/");
    }
}
```

OrangeHRMTest.java – Test Class with main()

```
// Login Page
LoginPage loginPage = new LoginPage(driver);

// Verify Login Page
if (loginPage.isLoginPageDisplayed()) {
    System.out.println("Login Page is displayed.");
} else {
    System.out.println("Login Page is NOT displayed.");
    driver.quit();
    return;
}

// Login steps
loginPage.enterUsername("Admin");
loginPage.enterPassword("admin123");
loginPage.clickLogin();
```

OrangeHRMTest.java – Test Class with main()

```
// Create HomePage object
HomePage homePage = new HomePage(driver);

// Verify Home Page
if (homePage.isDashboardDisplayed()) {
    System.out.println("Login successful. Dashboard is displayed.");
} else {
    System.out.println("Login failed or Dashboard not found.");
}

// Close the browser
driver.quit();
}
```

Output in Console

```
Login Page is displayed.
Login successful. Dashboard is displayed.
```

Page Object Model (POM)

POM PageFactory Difference

POM Without PageFactory	POM With PageFactory
Use By locators and driver.findElement() every time	Use @FindBy annotations
Elements are initialized manually	Elements are auto-initialized with PageFactory.initElements()
More verbose code	Cleaner, more readable code

Quiz



1) What is the main purpose of using the Page Object Model (POM)?

a) To manage database connections

b) To separate locators and actions from test scripts

c) To generate reports

d) To define test data

Answer : Option b)

Page Object Model (POM)

Quiz



2) In POM, what does each class typically represent?

a) A user

b) A test suite

c) A page or section of the web application

d) A data model

Answer : Option c)

Quiz



3) What is the primary benefit of using PageFactory in POM?

a) Slower execution

b) Reduces the need for driver setup

c) Initializes web elements at runtime using annotations

d) Avoids object-oriented programming

Answer : Option c)

Quiz



4) Which of the following is TRUE about POM without PageFactory?

a) It uses annotations like @FindBy

b) You manually initialize elements using `driver.findElement()`

c) It requires JUnit for execution

d) It's deprecated in Selenium 4

Answer : Option d)

Page Object Model (POM) and Data Driven Testing



Page Object Model (POM) and Data Driven Testing

Project Structure with excel file (.xlsx)

- Now we will combine Page Factory, Data-Driven Testing, and external data sources like Excel and flat files (CSV or .txt) - which is next level of automation framework.
- Project Structure

```
OrangeHRM_Excel_DDT/  
├── pages/  
│   ├── LoginPage.java  
│   └── HomePage.java ✓  
├── utils/  
│   └── ExcelUtils.java  
├── tests/  
│   └── OrangeHRMLoginTest.java  
├── testdata/  
│   └── loginData.xlsx
```

Page Object Model (POM) and Data Driven Testing

Required Dependency (Apache POI)

```
<dependencies>
  <dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi</artifactId>
    <version>5.2.3</version>
  </dependency>
  <dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>5.2.3</version>
  </dependency>
</dependencies>
```

Page Object Model (POM) and Data Driven Testing

testdata/loginData.xlsx – Sample Excel File

username	password
Admin	admin123
wronguser	wrongpass

pages/LoginPage.java – Page Object with PageFactory

```
package pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;

public class LoginPage {
    WebDriver driver;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }
}
```


pages/LoginPage.java – Page Object with PageFactory

```
@FindBy(name = "username")
WebElement usernameField;

@FindBy(name = "password")
WebElement passwordField;

@FindBy(css = "button[type='submit']")
WebElement loginButton;

public void login(String username, String password) {
    usernameField.clear();
    usernameField.sendKeys(username);
    passwordField.clear();
    passwordField.sendKeys(password);
    loginButton.click();
}
}
```

pages/HomePage.java

```
package pages;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;

public class HomePage {
    WebDriver driver;

    public HomePage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }
    @FindBy(xpath = "//h6[text()='Dashboard']")
    WebElement dashboardHeader;

    public boolean isDashboardDisplayed() {
        return dashboardHeader.isDisplayed();
    }
}
```

utils/ExcelUtils.java – Reads Excel File

```
package utils;

import org.apache.poi.ss.usermodel.*;
import java.io.FileInputStream;

public class ExcelUtils {

    public static String[][] readExcelData(String filePath, String sheetName) {
        String[][] data = null;

        try {
            FileInputStream fis = new FileInputStream(filePath);
            Workbook workbook = WorkbookFactory.create(fis);
            Sheet sheet = workbook.getSheet(sheetName);

            int rows = sheet.getPhysicalNumberOfRows();
            int cols = sheet.getRow(0).getPhysicalNumberOfCells();

            data = new String[rows - 1][cols]; // skip header
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

utils/ExcelUtils.java – Reads Excel File

```
        for (int i = 1; i < rows; i++) {  
            Row row = sheet.getRow(i);  
            for (int j = 0; j < cols; j++) {  
                data[i - 1][j] = row.getCell(j).toString();  
            }  
        }  
  
        workbook.close();  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
  
    return data;  
}  
}
```

tests/OrangeHRMLoginTest.java

```
package tests;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import pages.LoginPage;
import pages.HomePage;
import utils.ExcelUtils;

public class OrangeHRMLoginTest {
    public static void main(String[] args) {

        String[][] testData = ExcelUtils.readExcelData("testdata/loginData.xlsx", "Sheet1");

        for (String[] row : testData) {
            String username = row[0];
            String password = row[1];
            runLoginTest(username, password);
        }
    }
}
```

tests/OrangeHRMLoginTest.java

```
public static void runLoginTest(String username, String password) {  
    WebDriver driver = new ChromeDriver();  
    driver.manage().window().maximize();  
    driver.get("https://opensource-demo.orangehrmlive.com/");  
  
    LoginPage loginPage = new LoginPage(driver);  
    loginPage.login(username, password);  
  
    try {  
        Thread.sleep(2000); // Wait for page to load  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
  
    HomePage homePage = new HomePage(driver);  
  
    System.out.println("Testing credentials: " + username + " / " + password);  
}
```

tests/OrangeHRMLoginTest.java

```
try {  
    if (homePage.isDashboardDisplayed()) {  
        System.out.println(" Login successful.\n");  
    } else {  
        System.out.println("Login failed - Dashboard not found.\n");  
    }  
} catch (Exception e) {  
    System.out.println("Login failed - Exception: " + e.getMessage() + "\n");  
}  
  
driver.quit();  
}
```

Output in Console

Testing credentials: Admin / admin123
Login successful.

Testing credentials: wronguser / wrongpass
Login failed - Dashboard not found.

Page Object Model (POM) and Data Driven Testing

Project Structure with flat file (.csv)

- Now we will combine Page Factory, Data-Driven Testing, and external data sources like Excel and flat files (CSV or .txt) - which is next level of automation framework.
- Project Structure

```
OrangeHRM_CSV_DDT/  
├── pages/  
│   ├── LoginPage.java  
│   └── HomePage.java      ✓ (New)  
├── utils/  
│   └── CSVUtils.java  
├── tests/  
│   └── OrangeHRMLoginTest.java  
└── testdata/  
    └── loginData.csv
```


Page Object Model (POM) and Data Driven Testing

Required Dependency (Apache POI)

```
<dependencies>
  <dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi</artifactId>
    <version>5.2.3</version>
  </dependency>
  <dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>5.2.3</version>
  </dependency>
</dependencies>
```

Page Object Model (POM) and Data Driven Testing

testdata/loginData.csv – Sample CSV File

```
username,password  
Admin,admin123  
wronguser,wrongpass
```

pages/LoginPage.java – Page Object with PageFactory

```
package pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;

public class LoginPage {
    WebDriver driver;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }
}
```

pages/LoginPage.java – Page Object with PageFactory

```
@FindBy(name = "username")
WebElement usernameField;

@FindBy(name = "password")
WebElement passwordField;

@FindBy(css = "button[type='submit']")
WebElement loginButton;

public void login(String username, String password) {
    usernameField.clear();
    usernameField.sendKeys(username);
    passwordField.clear();
    passwordField.sendKeys(password);
    loginButton.click();
}
}
```

pages/HomePage.java

```
package pages;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;

public class HomePage {
    WebDriver driver;

    public HomePage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }
    @FindBy(xpath = "//h6[text()='Dashboard']")
    WebElement dashboardHeader;

    public boolean isDashboardDisplayed() {
        return dashboardHeader.isDisplayed();
    }
}
```

utils/ExcelUtils.java – Reads Excel File

```
package utils;

import java.io.*;
import java.util.*;

public class CSVUtils {

    public static List<String[]> readCSV(String filePath) {
        List<String[]> dataList = new ArrayList<>();

        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
            String line;
            boolean isHeader = true;

            while ((line = br.readLine()) != null) {
                if (isHeader) {
                    isHeader = false; // skip header row
                    continue;
                }
            }
        }
    }
}
```

utils/CSVUtils.java– Reads csv File

```
String[] parts = line.split(",");  
    dataList.add(parts);  
    }  
} catch (IOException e) {  
    e.printStackTrace();  
}  
  
return dataList;  
}  
}
```

tests/OrangeHRMLoginTest.java

```
package tests;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import pages.LoginPage;
import pages.HomePage;
import utils.CSVUtils;
import java.util.List;

public class OrangeHRMLoginTest {
    public static void main(String[] args) {
        System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");

        List<String[]> testData = CSVUtils.readCSV("testdata/loginData.csv");

        for (String[] row : testData) {
            String username = row[0];
            String password = row[1];
            runLoginTest(username, password);
        }
    }
}
```


tests/OrangeHRMLLoginTest.java

```
public static void runLoginTest(String username, String password) {  
    WebDriver driver = new ChromeDriver();  
    driver.manage().window().maximize();  
    driver.get("https://opensource-demo.orangehrmlive.com/");  
  
    LoginPage loginPage = new LoginPage(driver);  
    loginPage.login(username, password);  
  
    try {  
        Thread.sleep(2000); // Wait for login response  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
  
    HomePage homePage = new HomePage(driver);  
    System.out.println("Testing credentials: " + username + " / " + password);  
}
```

tests/OrangeHRMLoginTest.java

```
try {
    if (homePage.isDashboardDisplayed()) {
        System.out.println("Login successful.\n");
    } else {
        System.out.println("Login failed - Dashboard not found.\n");
    }
} catch (Exception e) {
    System.out.println("Login failed - Exception occurred: " + e.getMessage() + "\n");
}

driver.quit();
}
```

tests/OrangeHRMLLoginTest.java

Output in Console

Testing credentials: Admin / admin123
Login successful.

Testing credentials: wronguser / wrongpass
Login failed - Dashboard not found.

Drop Down List and its Methods



Drop Down List and its Methods

Introduction

- In Selenium, the Select class provides the implementation of the HTML SELECT tag.
- A Select tag provides the helper methods with select and deselect options.
- As Select is an ordinary class, its object is created by the keyword New and also specifies the location of the web element.
- The Select is a Webdriver class that can work with the HTML SELECT tag. It exposes several “Select By” and “Deselect By” type methods. You can apply these methods to change the value in the dropdown and multiple select fields
- This class is a part of **org.openqa.selenium.support.ui.Select** Package
- **Select Class Syntax in Selenium:**

Select selectobj = new Select(<WebElement Object>);

Introduction

```
WebElement dropdown = driver.findElement(By.id("Browsers"));
Select select = new Select(dropdown);

// Alternatively, you can shorten the same code as given below.

Select select = new Select(driver.findElement(By.id("Browsers")));
```

Drop Down List and its Methods

Different Select Class Methods

- The following are the most commonly used select class methods in selenium to deal with a drop-down list:



Drop Down List and its Methods

Different Select Class Methods

1. selectByVisibleText()

- This select class method is used to select one of the options in a drop-down box or an option among multiple selection boxes.
- It takes a parameter of String which is one of the values of Select element and it returns nothing.

Syntax

```
objSelect.selectByVisibleText("text");
```

Example:

```
Select objSelect =new Select(driver.findElement(By.id("search-box")));
```

```
objSelect.selectByVisibleText("Automation");
```


Different Select Class Methods

1. selectByVisibleText()

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Select Example By Visible Text</title>
  </head>
  <body>
    <html>
      <body>
        <select name="Browsers">
          <option value="0" selected>Select...</option>
          <option value="1">Chrome</option>
          <option value="2">FireFox</option>
          <option value="3">IE</option>
          <option value="4">Safari</option>
          <option value="5">Opera</option>
        </select>
      </body>
    </html>
  </body>
</html>
```

Drop Down List and its Methods

Different Select Class Methods

1. selectByVisibleText()

```
WebElement dropdown = driver.findElement(By.name("Browser  
s"));

Select select = new Select(dropdown);
select.selectByVisibleText("FireFox");
```

Drop Down List and its Methods

Different Select Class Methods

2. selectByIndex()

- This method is similar to 'selectByVisibleText', but the difference here is that the user has to provide the index number for the option rather than text.
- It takes the integer parameter which is the index value of Select element and it returns nothing.

Syntax

```
objSelect.selectByIndex(int);
```

Example:

```
Select objSelect =new Select(driver.findElement(By.id("search-box")));  
  
objSelect.selectByIndex(3);
```

Drop Down List and its Methods

Different Select Class Methods

1. selectByIndex()

```
WebElement dropdown = driver.findElement(By.name("Browsers"));

Select select = new Select(dropdown);
select.selectByIndex(4);
```

Drop Down List and its Methods

Different Select Class Methods

3. selectByValue()

- This method asks for the value of the desired option rather than the option text or an index. It takes a String parameter which is one of the values of Select element and it does not return anything.

Syntax

```
objSelect.selectByValue("text");
```

Example:

```
Select objSelect =new Select(driver.findElement(By.id("search-box")));  
  
objSelect.selectByValue("Auto");
```

Different Select Class Methods

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Select Example By Value</title>
  </head>
  <body>
    <html>
      <body>
        <select name="Browsers">
          <option value="0" selected>Select...</option>
          <option value="Chrome">Chrome</option>
          <option value="FireFox">FireFox</option>
          <option value="IE">IE</option>
          <option value="Safari">Safari</option>
          <option value="Opera">Opera</option>
        </select>
      </body>
    </html>
  </body>
</html>
```

Different Select Class Methods

3. selectByValue()

```
WebElement dropdown = driver.findElement(By.name("Browsers"));

Select select = new Select(dropdown);
select.selectByValue("Opera");
```

Drop Down List and its Methods

Different Select Class Methods

4. getOptions()

- This method gets all the options belonging to the Select tag. It takes no parameter and returns List<WebElements>.

Syntax

```
objSelect.getOptions();
```

Example:

```
Select objSelect = new Select(driver.findElement(By.id("Search-box")));
```

```
List <WebElement> elementCount = objSelect.getOptions();
```

```
System.out.println(elementCount.size());
```


Drop Down List and its Methods

Different Select Class Methods

DeSelect Methods

1. deselectByIndex Method : `objSelect.deselectByIndex(3);`

Use it when you want to deselect any option via its index. It accepts the index as an argument.

2. deselectByValue Method: `objSelect.deselectByValue("Opera");`

Use it when you wish to deselect all options using a value that matches the value in the dropdown options.

3. deselectByVisibleText Method : `objSelect.deselectByVisibleText("selenium");`

Use it to deselect all options by supplying the input text that matches the text in the dropdown options.

Different Select Class Methods

DeSelect Methods

4. deselectall Method: `objSelect.deselectAll();`

- Usage: Use it to turn off all the selected entries at once. But you can use this method if the `<Select>` tag has `<multiple>` attributes enabled in the HTML.
- Otherwise, it'll throw the `<NotImplemented>` exception. Hence, it's compulsory to have an attribute like `<multiple="multiple">` while defining the `<Select>` tag.

How to select multiple items in Drop Down using Select Class Method

- The multiple select attribute is a boolean expression. This method specifies that multiple options can be selected at once. These options vary for different operating systems and browsers.

isMultiple Method: `objSelect.isMultiple();`

`isMultiple()`: boolean – This method informs whether the Select element supports multiple selection options at the same time or not. This method accepts nothing and returns a boolean value (true/false).

Drop Down List and its Methods

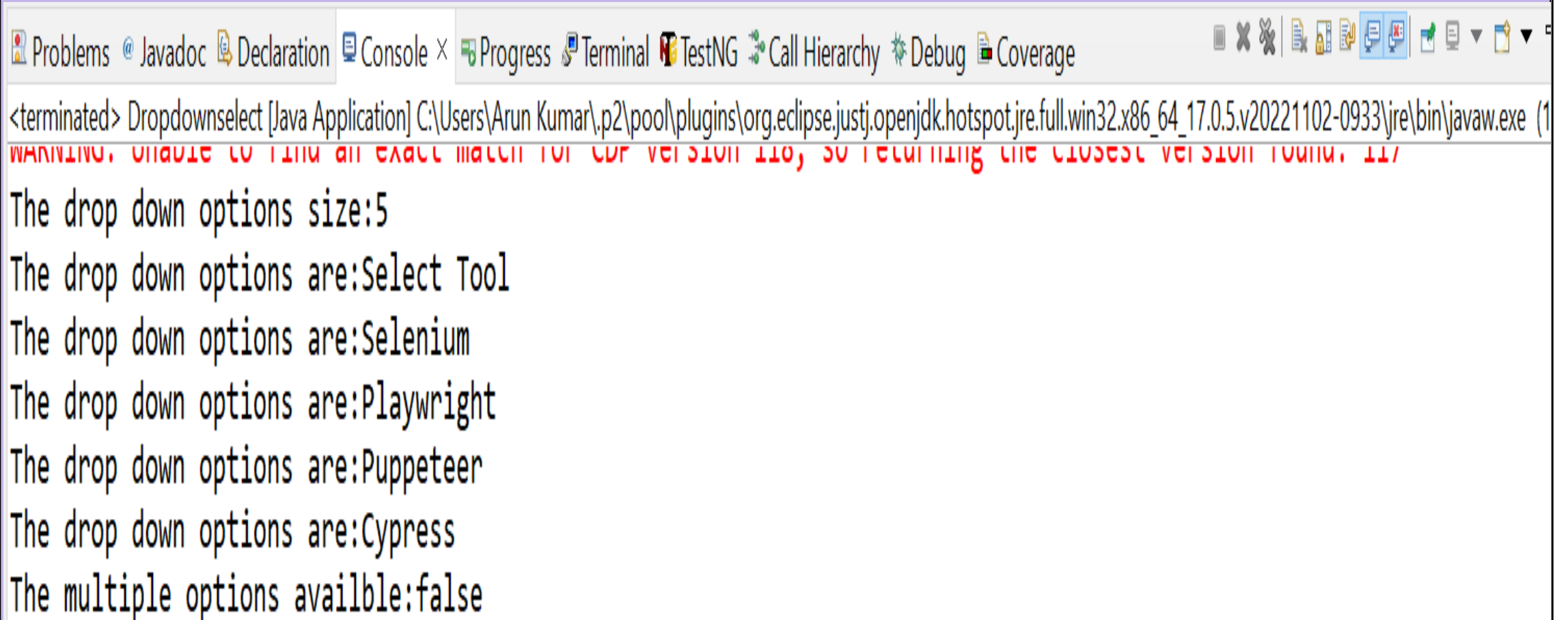
An Example Scenario

```
import org.openqa.selenium.support.ui.Select;
public class Dropdownselect {
    public static void main(String[] args) throws InterruptedException {
        // TODO Auto-generated method stub
        ChromeOptions options = new ChromeOptions();
        options.addArguments("--headless");
        WebDriver driver = new ChromeDriver(options);
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.get("https://leafground.com/select.xhtml");
        WebElement UIAutomation = driver.findElement(By.xpath("//select[@class='ui-selectonemenu']"));
        Select dropdown = new Select(UIAutomation);
        List<WebElement> list1 = dropdown.getOptions();
        System.out.println("The drop down options size:" +list1.size());
        for(WebElement ls : list1)
        {
            System.out.println("The drop down options are:" +ls.getText());
        }
        Boolean multiple = dropdown.isMultiple();
        System.out.println("The multiple options available:" +multiple);
        dropdown.selectByIndex(1); // by index
        dropdown.selectByVisibleText("Puppeteer"); //by visible text method
    }
}
```

Drop Down List and its Methods

An Example Scenario

Output:



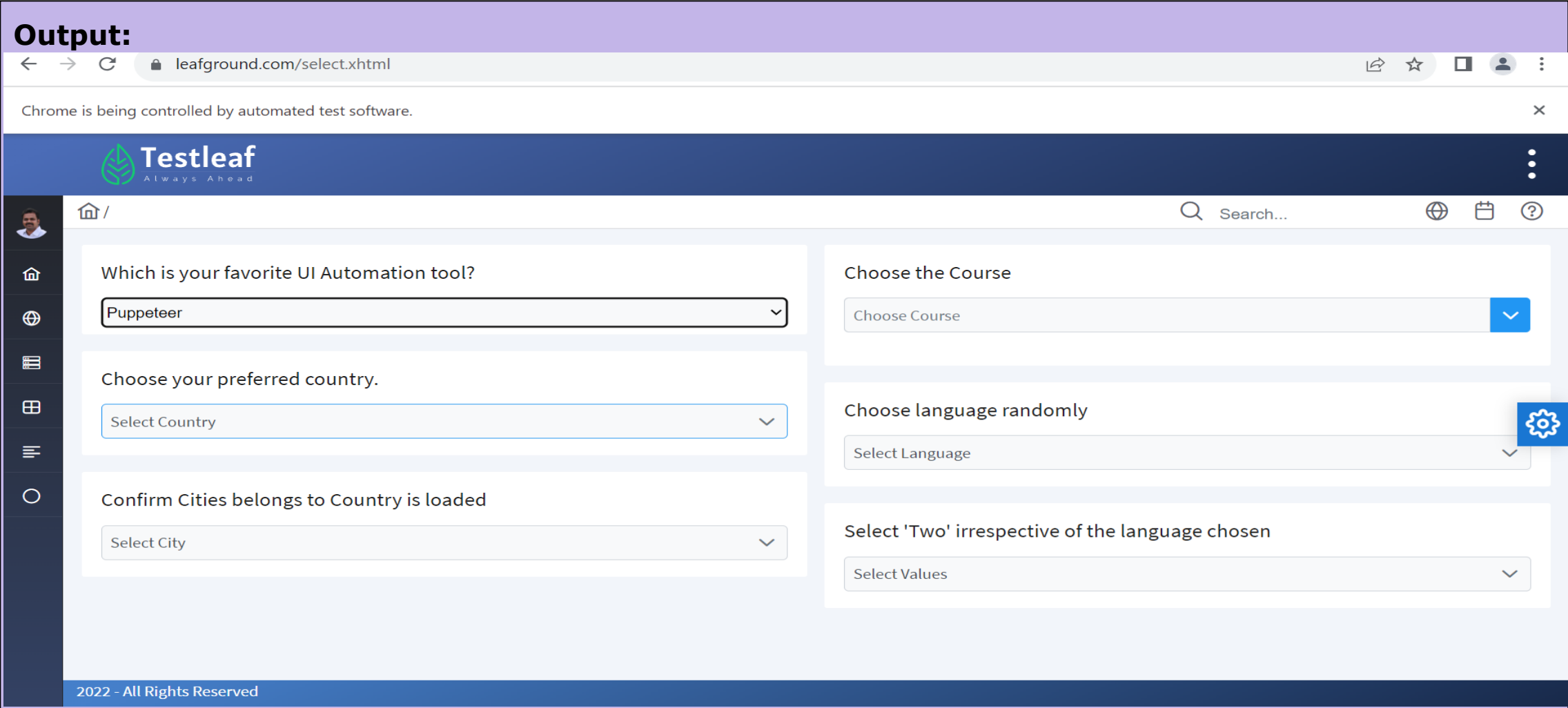
```

<terminated> Dropdownselect [Java Application] C:\Users\Arun Kumar\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.5.v20221102-0933\jre\bin\javaw.exe (1
WARNING: Unable to find an exact match for CDP version 110, so returning the closest version found, 117
The drop down options size:5
The drop down options are:Select Tool
The drop down options are:Selenium
The drop down options are:Playwright
The drop down options are:Puppeteer
The drop down options are:Cypress
The multiple options availble:false
  
```

Drop Down List and its Methods

An Example Scenario

Output:



Drop Down List and its Methods

How to select a drop-down list elements?

- Drop downs in a website could be created in several different ways.
- Some dropdowns are created using `<select>` HTML tag and some others are created using `<ul>` ,`<li>`, `<button>` and `<div>` tags.
- Some dropdowns are dynamic in nature which means after clicking or selecting any option, the drop downs values would populate accordingly.
- And in some cases, you need to mouse hover on an element in Selenium to see the drop-down options.
-

Drop Down List and its Methods

How to select a drop-down list elements?

- Selenium WebDriver provides `Select` class which can be used only for drop down created using `<select>` HTML tag.
- `Select` class has methods such as `selectByVisibleText()`, `selectByIndex()` and `selectByValue()` to select the desired option from dropdown.
- However, for `NonSelect` dropdowns, `Select` class cannot be used.

How to Select Dropdown in Selenium?

You can handle dropdown in Selenium using Select Class and without using Select Class. Below are 5 different methods that can be used to select value in dropdown in Selenium without using Select Class.

These methods are:

- 1.By storing all the options in List and iterating through it
- 2.By creating Custom Locator and without iterating the List
- 3.By using JavaScriptExecutor class
- 4.By using sendKeys
- 5.By using Actions Class

How to Select Dropdown in Selenium?

1.By storing all the options in List and iterating through it

- You can handle Dropdown in Selenium by storing all the options in the form of a List and then iterate through it

Drop Down List and its Methods

An Example Scenario

```
public class ListDemo {  
    public static void main(String[] args) {  
        ChromeOptions options = new ChromeOptions();  
        WebDriver driver = new ChromeDriver(options);  
        driver.manage().window().maximize();  
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));  
        driver.get("https://www.bstackdemo.com/");  
        driver.findElement(By.xpath("//select")).click();  
        //Method 1: By storing all the options in List and iterating through it  
        List<WebElement> allOptions = driver.findElements(By.cssSelector("select option"));  
        System.out.println("Options Size:" + allOptions.size());  
        String option = "Lowest to highest";  
        for (int i = 0; i < allOptions.size(); i++) {  
            if (allOptions.get(i).getText().contains(option)) {  
                allOptions.get(i).click();  
                System.out.println("clicked");  
                break;  
            }  
        }  
    }  
}
```

How to Select Dropdown in Selenium?

2. By creating Custom Locator and without iterating the List:

- In this method, you have to create custom Locator in Selenium, and then handle the drop down without iterating the List

```
//Method 2: By creating Custom Locator and without iterating the List  
String option1="Highest to lowest";  
WebElement dropdown =driver.findElement(By.xpath("//select/option[contains(text(),  
'" +option1+"' )]"));  
dropdown.click();
```

How to Select Dropdown in Selenium?

3. By using JavaScriptExecutor class:

- JavaScriptExecutor is an Interface that helps to execute JavaScript through Selenium WebDriver.
- You can use the executeScript method of JavaScriptExecutor interface to select an option from the drop down by using the value property of the element

//Method 3: By using JavaScriptExecutor class

```
WebElement option2 = driver.findElement(By.xpath("//select"));  
JavascriptExecutor js = (JavascriptExecutor) driver;  
js.executeScript("arguments[0].value='lowestprice'", option2);
```

How to Select Dropdown in Selenium?

4. By using sendKeys

- You can use sendKeys() method to select an option from a drop down by passing the element's value as sendKeys char sequence

//Method 4: By using sendKeys method

```
driver.findElement(By.xpath("//select")).sendKeys("highestprice");
```

How to Select Dropdown in Selenium?

5. By using Actions Class

- Some drop-downs need to be hovered to display all the options present in it and then click on it.
- Actions class of Selenium is used for handling keyboard and mouse events

//Method 5: By using Actions Class

```
WebElement ele = driver.findElement(By.xpath("//select"));
Actions action=new Actions(driver);
action.moveToElement(ele).perform();
WebElement customOption=driver.findElement(By.xpath("//option[text()='Lowest to highest']"));
customOption.click();
```

Drop Down List and its Methods



```
1 | <ol>
```

```
1 | <html>
2 |
3 | <head>
4 |   <title>Numbered List Example</title>
5 | </head>
6 |
7 | <body>
8 |   <h2>Ordered List with Numbers</h2>
9 |   <ol>
10 |     <li>JavaScript</li>
11 |     <li>Python</li>
12 |     <li>Java</li>
13 |     <li>C++</li>
14 |     <li>C#</li>
15 |   </ol>
16 | </body>
17 |
18 | </html>
```


How to Select Dropdown in Selenium?

```
//Locate the ordered list using its tag name
WebElement orderedList = driver.findElement(By.tagName(("ol")));

//Fetch all the list items
List<WebElement> listItems = orderedList.findElements(By.tagName("li"));

//Iterate through the list and print the contents
for (int i = 0; i < listItems.size(); i++) {
    System.out.println(listItems.get(i).getText());
}

//Close the main window
driver.quit();
```

How to Select Dropdown in Selenium?

```
WARNING: Unable to find an exact match for CDP v
JavaScript
Python
Java
C++
C#
```

How to Select Dropdown in Selenium?

```
List<WebElement> options = dropdown.getOptions();

// Extract the text of the options and store them in a list
List<String> optionTextList = new ArrayList<>();
for (WebElement option : options) {
    optionTextList.add(option.getText());
}

// Sort the list of option texts
Collections.sort(optionTextList);
```

Drop Down List and its Methods

Quiz



1) Which Selenium class is specifically designed to handle `<select>` drop-down menus?

a) DropdownHandler

b) WebSelect

c) Select

d) DropList

Answer : Option c)

Drop Down List and its Methods

Quiz



2) Which package does the Select class belong to?

a) org.openqa.selenium.support.ui

b) org.openqa.selenium.dropdown

c) org.openqa.selenium.support.webselect

d) org.openqa.selenium.support.select

Answer : Option a)

Drop Down List and its Methods

Quiz



3) What is the correct way to create a Select object for a drop-down WebElement?

a) `Select dropdown = new Select(driver);`

b) `Select dropdown = new Select("dropdownID");`

c) `Select dropdown = new
Select(driver.findElement(By.id("dropdown")));`

d) `WebElement dropdown = new Select();`

Answer : Option c)

Drop Down List and its Methods

Quiz



4) How do you select an option in a drop-down by its index?

a) `dropdown.selectByText("2");`

b) `dropdown.selectByVisibleIndex(2);`

c) `dropdown.selectByIndex(2);`

d) `dropdown.chooseByIndex(2);`

Answer : Option c)

Logging – Log4j



Logging – Log4j

Logging

- Logging is a process that takes applications to a newer level with information logs on how the applications may or may not have performed/executed. It gives an exact idea of software performance, including any shortcomings.
- Log4j in Selenium is one such logging framework that helps in gathering information in the form of logs or log files.

Logging – Log4j

Log4j

- Log4j is a logging framework written in Java that provides an easy way for logging in Selenium.
- In a nutshell, the framework gives out information about everything that goes on during the software execution.
- Log4j also provides insight into anything that may have gone wrong during software execution or automation.
- Overall, Log4j documents the output in the form of logs that can be examined later for purposes such as auditing small and large-scale Selenium projects.

Logging – Log4j

Log4j Components

The Log4j logging framework comprises the following components:

1. Logger
2. Appenders
3. Layout

Logging – Log4j

1. Logger

The function of the logger in Log4j is basically storing and capturing all the necessary logging information that will be generated using the framework.

- **Logger Class** – To fully use the logger, **create an instance for a logger class where all the generic methods will be at the user's disposal**, required to use Log4j.
- **Log Levels** – These are the methods that will be used to print the log messages. There are primarily only a few log levels that are used in a script.

Logging – Log4j

Logger levels -Log4j

1. **ALL** – This level will prioritize and include everything in the logs.
2. **ERROR** – This level will show messages that inform users about error events that may not stop the application.
3. **WARN** – This level will show information regarding warnings, that may not stop the execution but may still cause problems.
4. **DEBUG** – This level will log debugging information.
5. **INFO** – This level will log the progress of the application.
6. **FATAL** – This will print information critical to the system that may even crash the application.

Logger levels -Log4j2

OFF

FATAL

ERROR

WARN

INFO

DEBUG

TRACE

ALL

Syntax

To start logging messages using this basic configuration, all you need to do is obtain a Logger instance using the LogManager class:

```
private static Logger logger = LogManager.getLogger(MyService.class);
```

Then you can use the logger object with methods corresponding to the log level you want:

```
logger.error("This is an error message");
```

Logging – Log4j

Appenders

- The appender basically grabs information from the logger and writes log messages to a file or any other storage location.
- The following are some of the appenders one can use for Log4j:
 1. **FileAppender** – This will append the log messages to a file.
 2. **RollingFileAppender** – It will perform the same function as FileAppender, but users will be able to specify the maximum file size. Once the limit is exceeded, the appender will create another file to write the messages.
 3. **DailyRollingFileAppender** – It specifies the frequency by which the messages are to be written to the file.
 4. **ConsoleAppender** – In this, the appender will simply write the log messages in the console.

Layout

- The layout is where the format in which log messages will appear is decided.
- There are several layouts one can use for log messages:
 1. **Pattern Layout** – The user must specify a conversion pattern based on which the logs will be displayed. Otherwise, it takes the default conversion pattern in case of “no pattern specified”.
 2. **HTML Layout** – In this layout, the format of logs will be in the form of an HTML table.
 3. **XML Layout** – This will show the logs in an XML format

Logging – Log4j

Dependency to be added in pom.xml -log4j

```
<!-- https://mvnrepository.com/artifact/org.apache.logging.log4j/log4j-core -->
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-core</artifactId>
  <version>2.19.0</version>
</dependency>
<!-- https://mvnrepository.com/artifact/org.apache.logging.log4j/log4j-api -->
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-api</artifactId>
  <version>2.19.0</version>
</dependency>
```

Logging – Log4j

Logging Execution steps

Follow the steps below to successfully run Log4j with Selenium Webdriver:

1. Write an automation script
2. After creating the script, create a log4j.properties file and specify the root logger, appended, and layout information in the file.
3. Import log4j dependencies like Logger, PropertyConfigurator, and add them to the script along with the logger class.
4. Add the messages that will be displayed in the log file

Logging – Log4j

log4j2.properties file

- **Create log4j2.properties file i n src/main/resources/log4j2.properties**

Configuration name

name = PropertiesConfig

Log file output folder

property.filename = logs

Define appenders

appenders = console, file

Console Appender

appender.console.type = Console

appender.console.name = STDOUT

appender.console.layout.type = PatternLayout

appender.console.layout.pattern = [%-5level] %d{yyyy-MM-dd HH:mm:ss.SSS} [%t] %c{1} - %msg%n

Logging – Log4j

log4j2.properties file

- **Create log4j2.properties file in src/main/resources/log4j2.properties**

File Appender

appender.file.type = File

appender.file.name = LOGFILE

appender.file.fileName = \${filename}/logs.log

appender.file.layout.type = PatternLayout

appender.file.layout.pattern = [%-5level] %d{yyyy-MM-dd HH:mm:ss.SSS} [%t] %c{1} - %msg%n

Root Logger Configuration

rootLogger.level = debug

rootLogger.appenderRefs = stdout, file

rootLogger.appenderRef.stdout.ref = STDOUT

rootLogger.appenderRef.file.ref = LOGFILE

tests/OrangeHRMLoginTest.java - updated with log4j

```
package tests;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import pages.LoginPage;
import pages.HomePage;
import utils.CSVUtils;
import io.github.bonigarcia.wdm.WebDriverManager;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

import java.util.List;

public class OrangeHRMLoginTest {
    public static Logger log = LogManager.getLogger(OrangeHRMLoginTest.class);

    public static void main(String[] args) {
        WebDriverManager.chromedriver().setup();

        List<String[]> testData = CSVUtils.readCSV("testdata/loginData.csv");
```

tests/OrangeHRMLoginTest.java - updated with log4j

```
for (String[] row : testData) {  
    String username = row[0];  
    String password = row[1];  
    runLoginTest(username, password);  
}  
}  
  
public static void runLoginTest(String username, String password) {  
    WebDriver driver = new ChromeDriver();  
    driver.manage().window().maximize();  
    log.info("Launching browser and navigating to OrangeHRM");  
    driver.get("https://opensource-demo.orangehrmlive.com/");  
  
    try {  
        LoginPage loginPage = new LoginPage(driver);  
        log.info("Attempting login with username: " + username);  
        loginPage.login(username, password);  
  
        Thread.sleep(2000); // Wait for page load (demo purpose)
```

tests/OrangeHRMLoginTest.java - updated with log4j

```
HomePage homePage = new HomePage(driver);

if (homePage.isDashboardDisplayed()) {
    System.out.println("Login successful for user: " + username + "\n");
    log.info("Login successful for user: " + username);
} else {
    System.out.println("Login failed for user: " + username + " - Dashboard not found.\n");
    log.warn("Login failed for user: " + username + " - Dashboard not found");
}

} catch (Exception e) {
    System.out.println("Exception occurred while testing user: " + username + "\n");
    log.error("Exception occurred for user: " + username + " - " + e.getMessage());
    e.printStackTrace();
} finally {
    driver.quit();
    log.info("Browser closed for user: " + username);
}

}
```

tests/OrangeHRMLoginTest.java

Output in Console

12:01:35.432 [main] INFO tests.OrangeHRMLoginTest - Launching browser and navigating to OrangeHRM

12:01:37.145 [main] INFO tests.OrangeHRMLoginTest - Attempting login with username: Admin

Login successful for user: Admin

12:01:41.785 [main] INFO tests.OrangeHRMLoginTest - Browser closed for user: Admin

12:01:42.012 [main] INFO tests.OrangeHRMLoginTest - Launching browser and navigating to OrangeHRM

12:01:43.751 [main] INFO tests.OrangeHRMLoginTest - Attempting login with username: wronguser

Login failed for user: wronguser - Dashboard not found.

12:01:47.432 [main] WARN tests.OrangeHRMLoginTest - Login failed for user: wronguser - Dashboard not found

12:01:47.567 [main] INFO tests.OrangeHRMLoginTest - Browser closed for user: wronguser

tests/OrangeHRMLoginTest.java

logs/logs.log (File Log Output)

```
[INFO ] 2025-08-04 12:01:35.432 [main] OrangeHRMLoginTest - Launching browser and navigating to Ora
[INFO ] 2025-08-04 12:01:37.145 [main] OrangeHRMLoginTest - Attempting login with username: Admin
[INFO ] 2025-08-04 12:01:41.785 [main] OrangeHRMLoginTest - Login successful for user: Admin
[INFO ] 2025-08-04 12:01:41.789 [main] OrangeHRMLoginTest - Browser closed for user: Admin

[INFO ] 2025-08-04 12:01:42.012 [main] OrangeHRMLoginTest - Launching browser and navigating to Ora
[INFO ] 2025-08-04 12:01:43.751 [main] OrangeHRMLoginTest - Attempting login with username: wrongus
[WARN ] 2025-08-04 12:01:47.432 [main] OrangeHRMLoginTest - Login failed for user: wronguser - Dash
[INFO ] 2025-08-04 12:01:47.567 [main] OrangeHRMLoginTest - Browser closed for user: wronguser
```



Quiz



1) Which of the following is NOT a valid Log4j logging level?

a) INFO

b) DEBUG

c) WARN

d) PRINT

Answer : Option d)

Quiz



2) What is the correct method to create a Log4j Logger instance?

a) `Logger log = new Logger();`

b) `Logger log = Logger.getLogger(Logger.class);`

c) `Logger log = Logger.create();`

d) `Logger log = LogManager.get();`

Answer : Option b)

Quiz



3) In a log4j.properties file, what does this line do?

log4j.rootLogger=DEBUG, file

a) Sets file name to DEBUG

b) Logs only file-level logs

c) Sets root log level to DEBUG and output to a file

d) Disables console logging

Answer : Option c)

Quiz



4) What is an Appender in Log4j?

a) A method to close logging

b) A tool to modify logs

c) A component that defines where log messages go (e.g., console, file)

d) A formatter for test steps

Answer : Option c)

Quiz



5) Which of the following layouts is commonly used in Log4j for readable output?

a) Simple Layout

b) HTML Layout

c) Pattern Layout

d) All the above

Answer : Option d)

THANK YOU